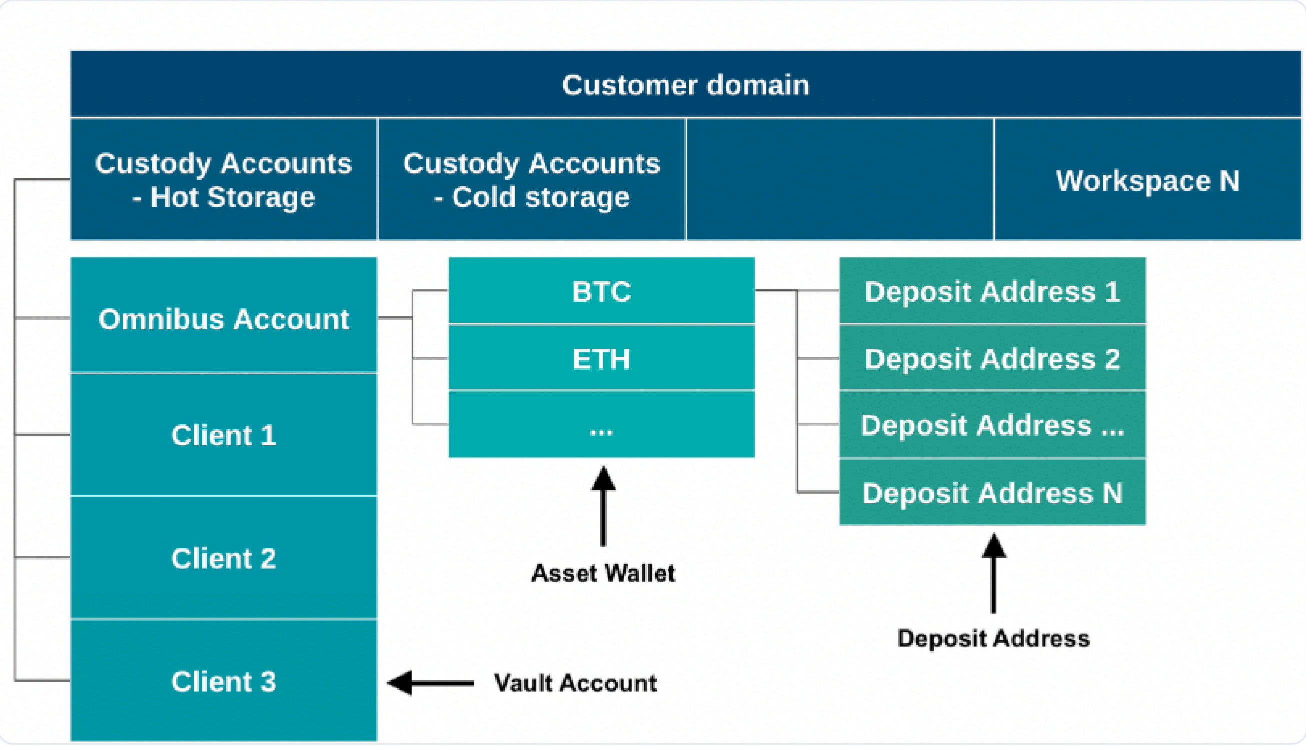
Account and wallet structure

Updated 4 months ago

Overview

In general, the Fireblocks platform consists of workspaces, vault accounts, asset wallets, and deposit addresses.

This diagram shows the overall account structure in Fireblocks and the underlying wallet structure.

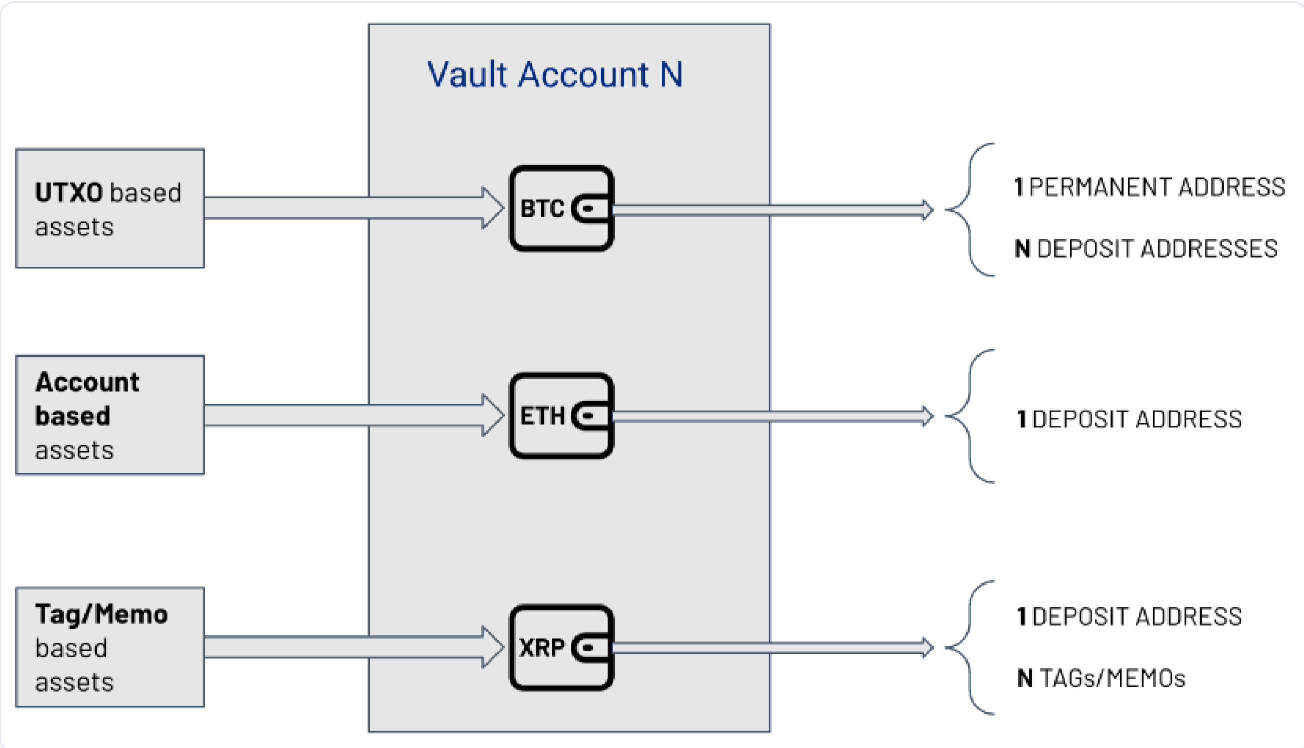


At the top level, your various Fireblocks workspaces are contained within the logical group called the *Customer Domain*. These workspaces may be *hot workspaces*, which contain hot wallets, or *cold workspaces*, which contain cold wallets.

In your Fireblocks workspace, you can create and manage multiple vault accounts, which contain your asset wallets.

Important

1. Account-based assets without tag/memo support, such as ETH, can only generate one deposit address.
2. Account-based assets with tag/memo support, such as XRP, can generate one or more deposit addresses. Each deposit address has the same on-chain address. However, they differ by their tag/memo.



Types of workspaces

There are two types of Fireblocks workspaces: Mainnet and Testnet. Any additional workspaces utilized by Fireblocks for testing or other purposes will not be referenced in this article or other design documents.

- **Mainnet workspaces:** These environments are used for staging and production purposes. Mainnet workspaces are only connected to Mainnet nodes of all supported blockchains.
- **Testnet workspaces:** These environments are used for sandbox and development purposes. Testnet workspaces are only connected to Testnet nodes of all supported blockchains.

Learn more about the differences between Mainnet and Testnet workspaces.

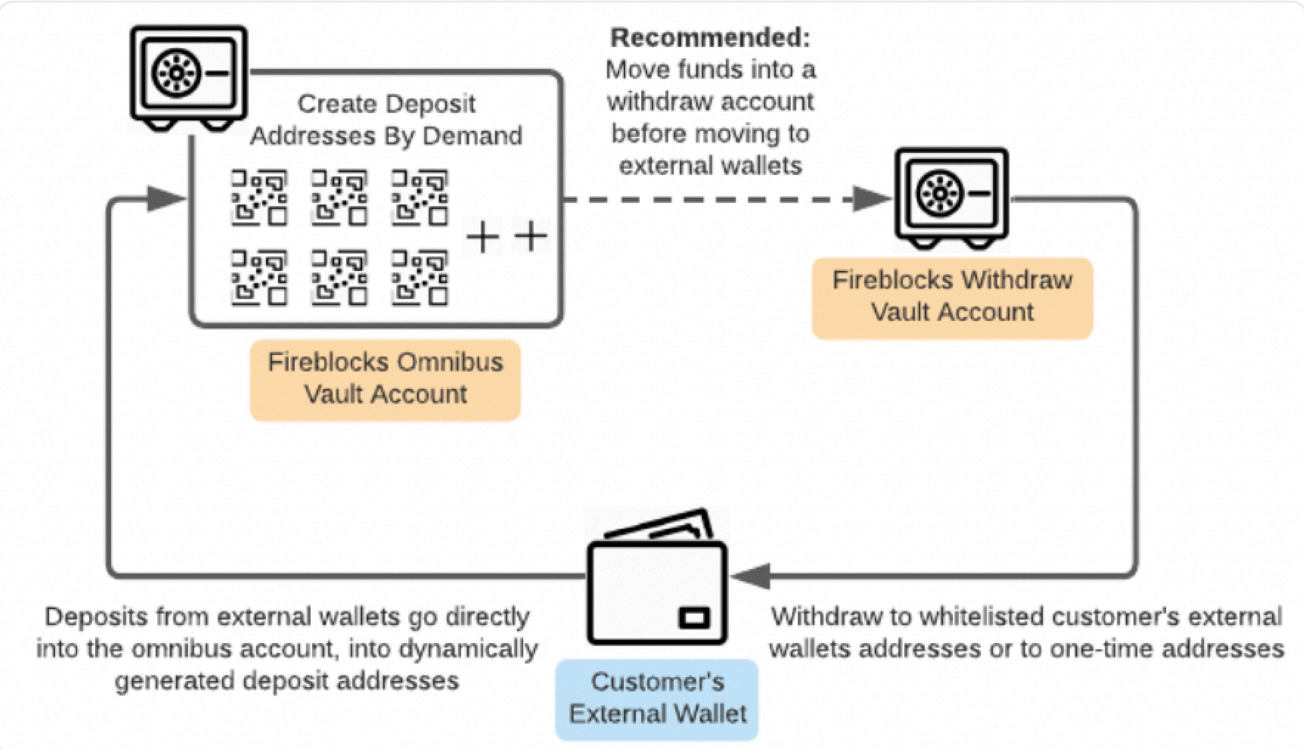
Types of structures

	Omnibus	Segregated
Advantages	• Internal transactions do not incur fees, settled on a private ledger off-chain • Simplifies account management, apply needed logic on internal private ledger • Inherently provides privacy for customers • Will work seamlessly with Enterprise Core Banking ledgers	• Using blockchain as a single source of truth ledger, removes the need for reconciliation • Settlements are inherently atomic (on-chain), reduces complexity and counterparty risk • Simplifes tracking and auditing
Disadvantages	• Requires extra measures to apply reconciliation on private ledger vs. blockchain nodes • Omnibus structure on Account-based networks, e.g; Ethereum, requires an intermediate account level • Complex protocols, e.g; Smart Contracts, may hinder reconciliation efforts	• In case most settlements are internal, requires on-chain transaction and corresponding fees • Most public blockchain networks offer lower Transactions Per Second volume then is required by Enterprises • Requires extra measures to mitigate customer privacy concerns

Omnibus structure

The omnibus structure consists of a central vault account in addition to vault accounts for each end client. Funds are deposited into the individual vault accounts and then swept to the central vault account, where the funds can be invested.

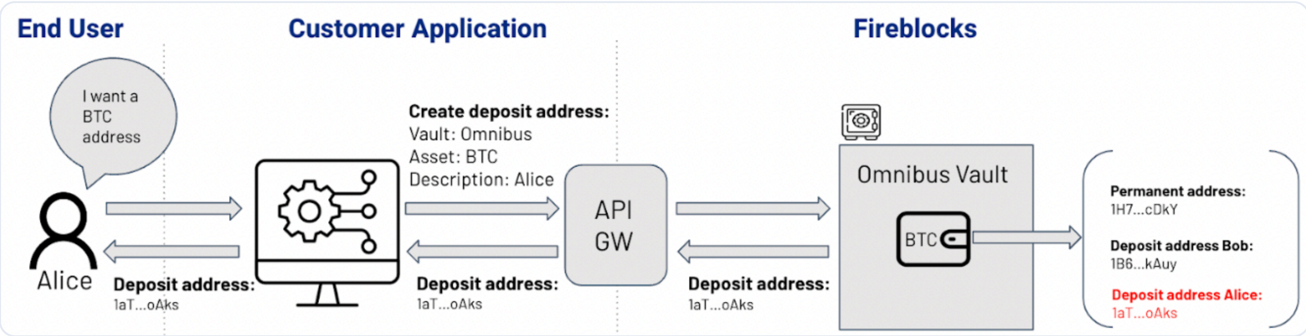
The diagram below shows a high-level representation of the omnibus vault account structure in Fireblocks for all asset types that are not account-based assets without tag/memo support.



- Omnibus and segregated account structures are independent of whether you have a hot or cold workspace. Both account structures can be implemented in either workspace type.
- Workspaces can contain both an omnibus and segregated account models. There is no limitation for a workspace to contain one specific account model.

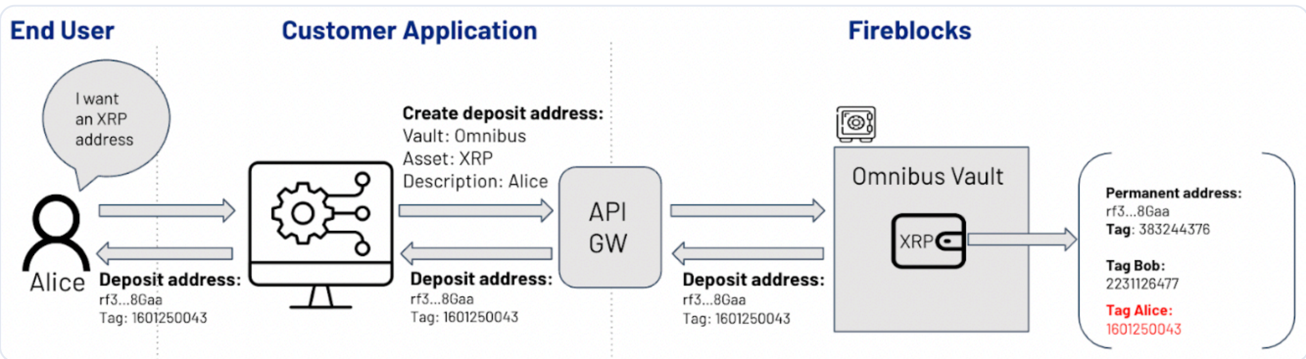
Below are some diagrams representing different flows for both Omnibus and Segregated account structures:

Omnibus address creation (UTXO)



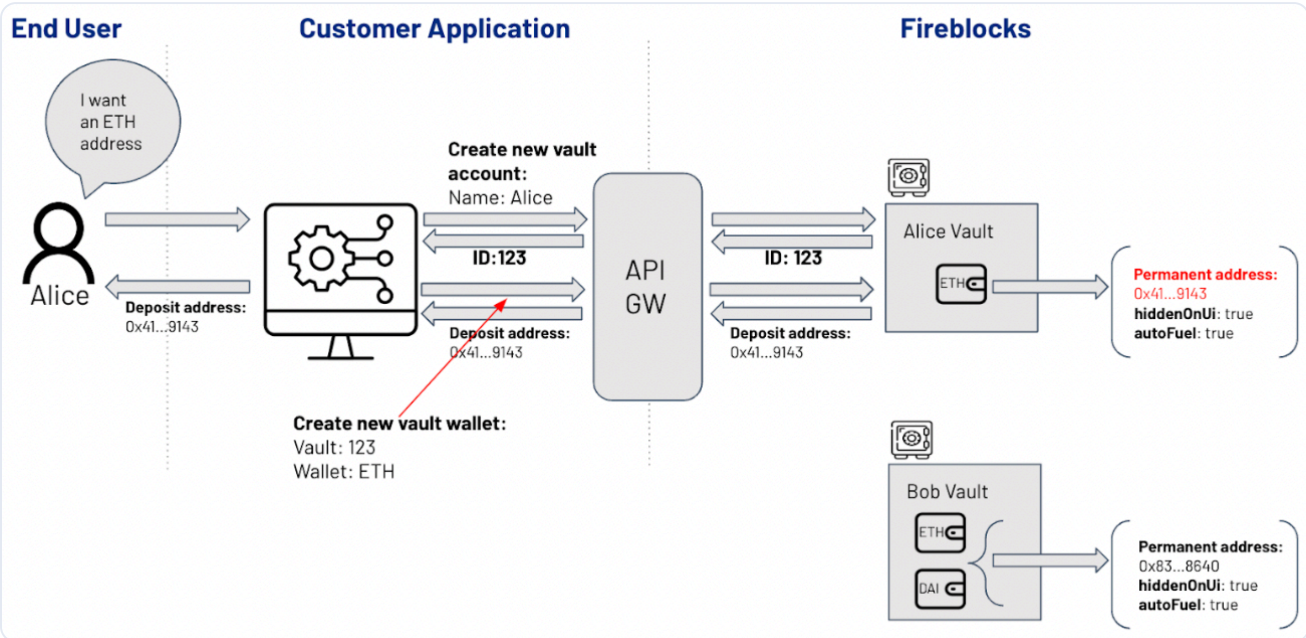
1. The end user, Alice, asks to deposit a UTXO asset, such as Bitcoin (BTC).
2. The customer's front-end application creates a new deposit address in the Omnibus vault account.
3. This deposit address is assigned to Alice and mapped accordingly within the customer's private ledger.
4. Alice receives a notification from the front-end application with her assigned deposit address, where she can start depositing funds.

Omnibus address creation (account-based with tag/memo support)



customer's private ledger.
4. Alice receives a notification from the front-end application with her assigned deposit address, where she can start depositing funds.

Omnibus Address Creation (account-based)

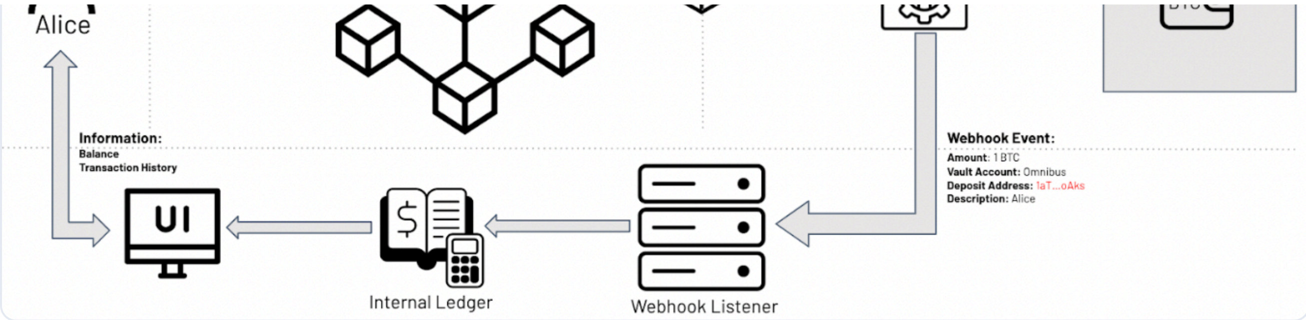


1. The end user, Alice, asks to deposit an account-based asset, such as Ethereum (ETH).
2. The customer's front-end application creates a new intermediate vault account and an asset wallet with a deposit address within it.
3. This deposit address is assigned to Alice and mapped accordingly within the customer's private ledger.
4. Once mapped and credited, Alice receives a notification from the front-end application with her assigned deposit address, where she can start depositing funds.

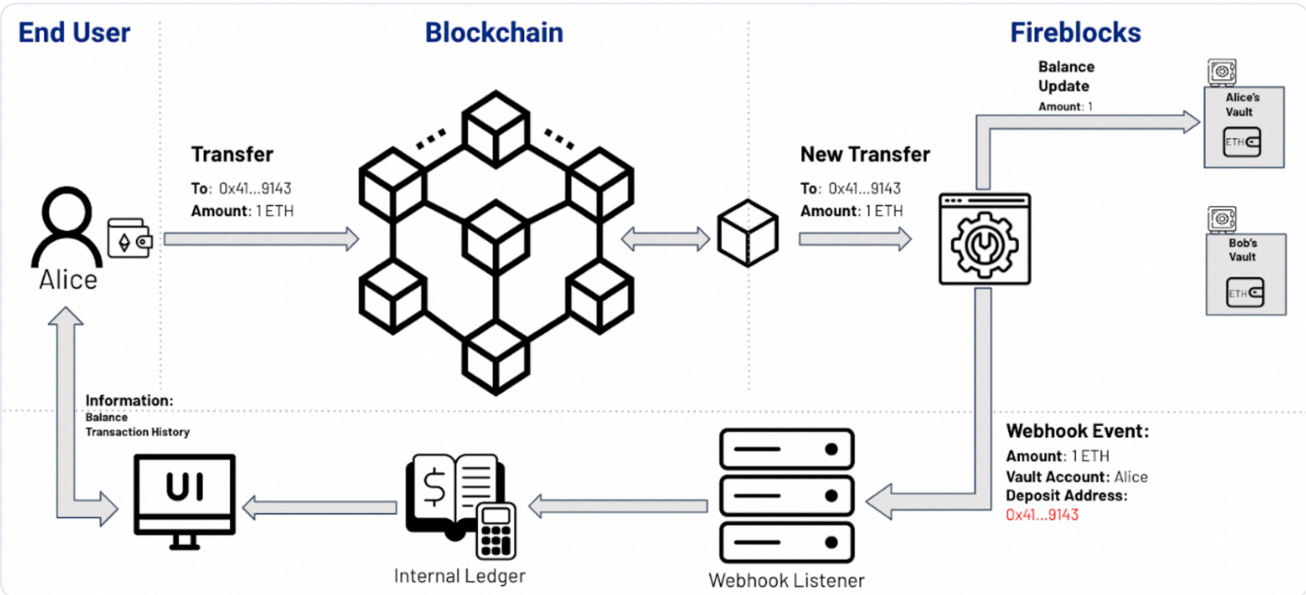
Deposit Flows

The diagrams below show the deposit flow for both asset types within the Omnibus account structure.

Omnibus architecture for UTXO-based assets



Omnibus architecture for account-based assets



1. Alice deposits funds into her dedicated deposit address.
2. A new incoming transfer appears in Fireblocks and, once confirmed on-chain, the wallet's balance is updated.
3. A webhook event is triggered and sent to the customer's webhook listener component.
4. The webhook listener processes the deposit event and updates the internal ledger accordingly.
5. The customer's front end updates Alice's current balance and transaction history to reflect the newly accredited deposit.

Withdrawal Flows

Due to various protocol limitations and depending on the implementation details of the blockchain protocol itself, Fireblocks highly recommends having more than one withdrawal vault account and performing round-robin withdrawals between them.

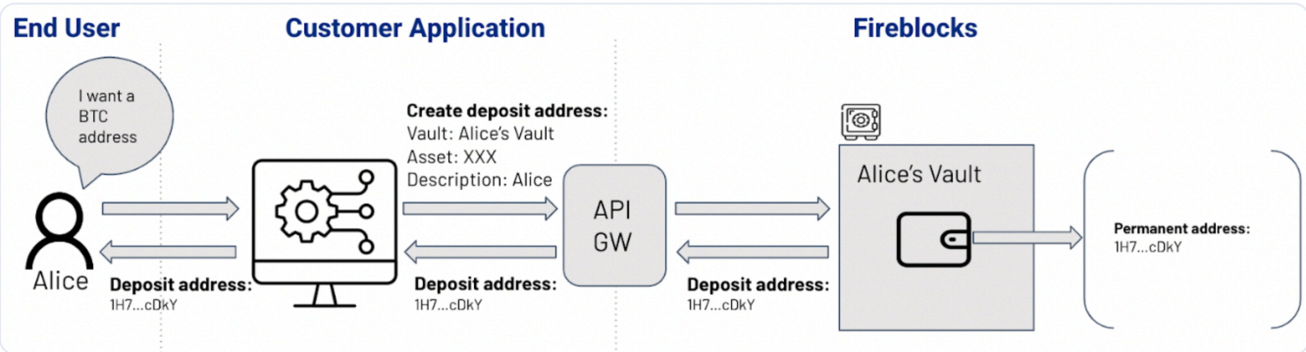
where the same unconfirmed input is used in more than 25 subsequent transactions. This is due to Bitcoin Core's implementation of the default maximum limit of unconfirmed transaction chains.

Segregated structure

The segregated structure consists of individual vault accounts for each end client. Funds are stored in and invested from these individual accounts.

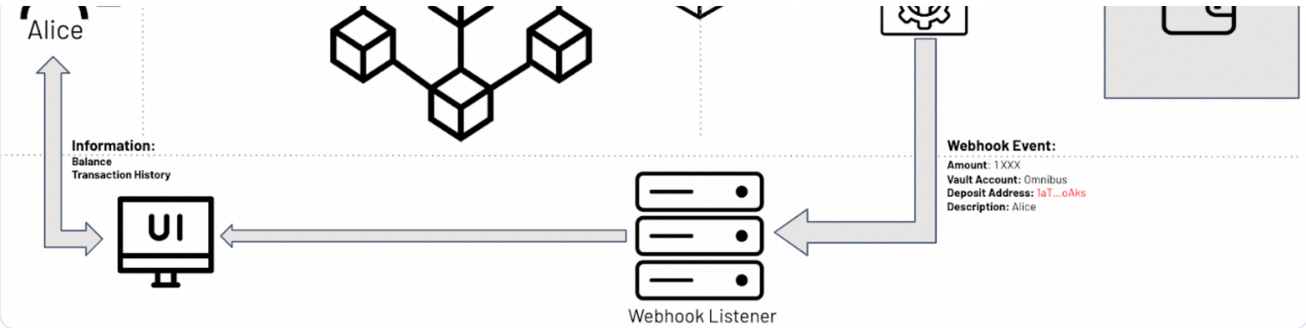
Address creation flows

There's no difference between UTXO and account-based assets when dealing with the segregated account structure. For both, the process is as follows:



1. The end user, Alice, asks to deposit some assets.
2. The customer's front-end application creates a new vault account if it does not exist or reuses an existing vault account dedicated to Alice. Then the asset wallet is created with a deposit address within the vault account.
3. Alice receives a notification from the front-end application with her assigned deposit address, where she can start depositing funds.

Deposit Flows



1. Alice deposits funds into her dedicated deposit address.
2. A new incoming transfer appears in Fireblocks and, once confirmed on-chain, the wallet's balance is updated.
3. A webhook event is triggered and sent to the customer's webhook listener component.
4. The webhook listener processes the deposit event and updates the front-end application.
5. The customer's front end updates Alice's current balance and transaction history to reflect the newly accredited deposit.

Notice that, in this flow, there is no private ledger involved since the blockchain is used as a ledger, and wallet balances are dedicated to a specific account.

Withdrawal Flows

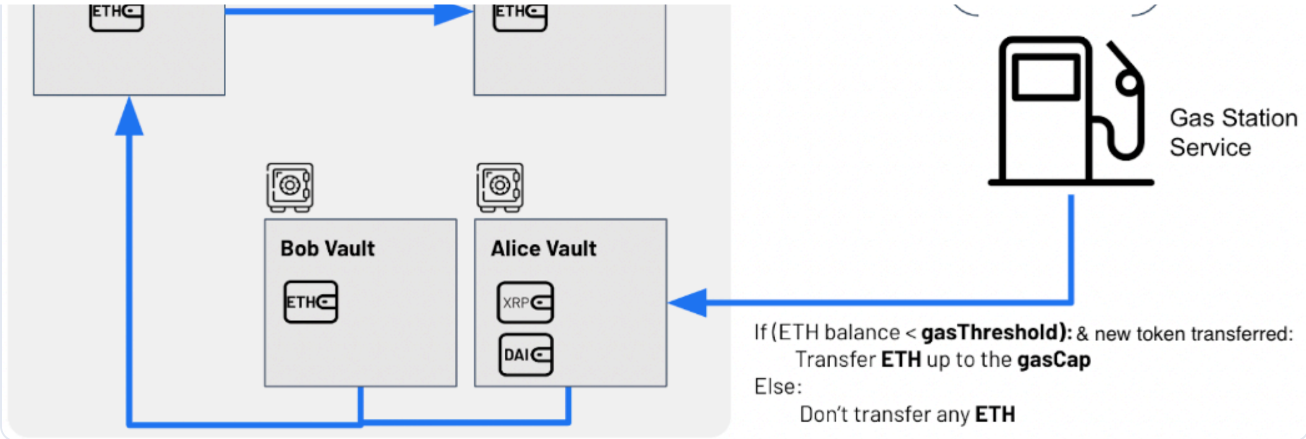
Assuming there are no restrictions or substantial withdrawal volumes (in terms of transaction count), withdrawal from the segregated account structure should be straightforward.

Fireblocks Gas Station

For a detailed explanation, see Fireblocks Gas Station.

The Fireblocks Gas Station can be used to support an account-based sweeping process where the customer periodically sweeps all the account-based assets into the omnibus account. Learn more about omnibus sweeping.

The following diagram depicts an end-to-end flow of the Fireblocks Gas Station during an EVM token deposit and credit process for an end-user.



Was this article helpful?

Yes No

ON THIS PAGE